Pick2Light (P2L) — System Specification & Architecture

© 2026 ThangLam. All rights reserved.

Table of Contents

1. Overview
2. System Architecture
3. Component: P2L .NET Service
   - 3.1 P2L.Shared
   - 3.2 P2L.Hardware
   - 3.3 P2L.Core
   - 3.4 P2L.Mqtt — MQTT Bridge
   - 3.5 P2L.CLI
4. MQTT Protocol Reference
   - 4.1 Topics
   - 4.2 Command Messages (Subscribe)
   - 4.3 Event Messages (Publish)
5. Aioi Hardware Protocol
6. Component: P2L Web Client
7. Component: SDK
   - 7.1 Python SDK
   - 7.2 Node.js SDK
8. Configuration Reference
   - 8.1 P2L appsettings.json
   - 8.2 Client p2l_config.json
9. Data Flow — End to End
10. Startup Sequence
11. Build & Deployment

1. Overview

Pick2Light (P2L) is a warehouse automation system that controls **Aioi Systems light modules** (AIW/AW series) to guide operators during picking operations. When an item needs to be picked, the corresponding shelf module lights up with a quantity display; the operator picks and presses CONFIRM; the system records the confirmation and extinguishes the light.

Key capabilities:

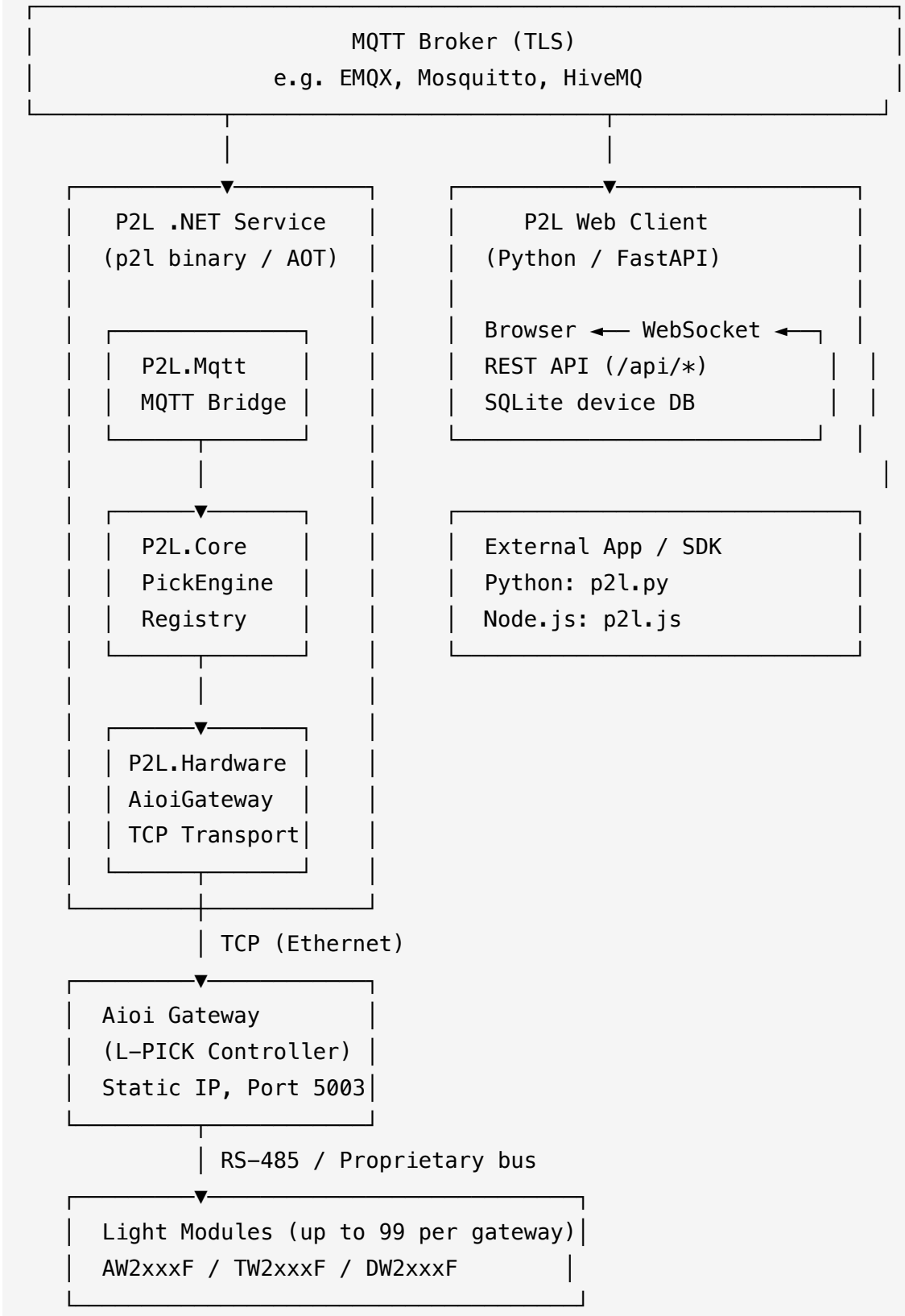
- Control individual light modules: color, LED mode (solid/flash), display digits
- Batch picking across multiple modules simultaneously
- Real-time confirmation events via hardware button press
- MQTT bridge for integration with WMS, ERP, and custom applications
- Web dashboard for monitoring and manual control
- SDK for Python and Node.js automation

Hardware: Aioi Systems L-PICK Controller (AIW/AW2 series), connected via TCP/Ethernet.

Platform: .NET 10 (Native AOT), cross-platform — Windows, Linux (systemd), macOS.

---

2. System Architecture


MQTT Broker (TLS)
e.g. EMQX, Mosquitto, HiveMQ
P2L .NET Service
(p2l binary / AOT)
P2L.Mqtt
MQTT Bridge
P2L.Core
PickEngine
Registry
P2L.Hardware
AioiGateway
TCP Transport
P2L Web Client
(Python / FastAPI)
Browser ⟵ WebSocket
REST API (/api/*)
SQLite device DB
External App / SDK
Python: p2l.py
Node.js: p2l.js
TCP (Ethernet)
Aioi Gateway
(L-PICK Controller)
Static IP, Port 5003
RS-485 / Proprietary bus
Light Modules (up to 99 per gateway)
AW2xxxF / TW2xxxF / DW2xxxF

3. Component: P2L .NET Service

The core service is a single AOT-compiled binary (`p2l`) composed of five .NET projects.

```
src/
├── P2L.Shared/       # Enums, DTOs, shared constants
├── P2L.Hardware/     # TCP transport, Aioi protocol encoder/decoder
├── P2L.Core/         # Business logic: picking engine, registry, scan
├── P2L.Mqtt/         # MQTT bridge: commands in, events out
└── P2L.CLI/          # Entry point, DI wiring, Spectre.Console dashboard
```

3.1 P2L.Shared

Shared types used across all layers. No external dependencies.

Enums

Enum	Values
`LightColor`	`Off, Red, Green, Blue, Yellow, Cyan, Magenta, White`
`LightState`	`Off(0x1), On(0x2), Flash(0x3), HighFlash(0x4), NoChange(0xF)`
`ModuleStatus`	`Unknown, PowerOn, Standby, Operating, Operating2, Completed, Shortag`

Enum	Values
`CompletionStatus`	`Normal(0), Abnormal(1), Shortage(10)`

Key Models

Model	Purpose
`PickingInstruction`	Command to light up a module: address, quantity, color, LED mode, block number, label
`LightModuleState`	Full state snapshot of a module: address, status, color, display, timestamps
`ConfirmationEvent`	Hardware confirmation: address, status, adjusted quantity, sequence number
`ModuleParameters`	Color + LED/SEG/BUZ mode triple for mode-array encoding

3.2 P2L.Hardware

Handles all TCP communication with the Aioi gateway.

`AioiGateway`

- Owns a `TcpTransport` and manages connection lifecycle
- Exposes `ConnectionChanged: event Action<bool>` — fires on connect/disconnect
- Exposes `IsConnected: bool`
- Routes decoded packets to registered command handlers
- Sends heartbeat packets via `HeartbeatService`

`TcpTransport`

- Maintains a persistent `TcpClient` to the gateway
- Auto-reconnect with exponential back-off

- Reads from `NetworkStream` into a ring buffer; parses `AioiPacket` frames
- Thread-safe send queue via `System.Threading.Channels`

`HeartbeatService`

- Sends `Z` (initialize) packets periodically to keep the TCP connection alive and prevent gateway timeout

`DeviceScanService`

- `ScanAsync(startAddr, endAddr, maxConcurrency, progress, ct)` — probes addresses 1–99 (configurable)
- Sends `Az{addr}` (status inquiry) for each address; records responding modules
- Registers found devices in `LightModuleRegistry` → triggers `StateChanged` → MQTT publish

Aioi Packet Frame

All Ethernet communication uses this ASCII frame structure:

```
┌─────┬─────────────┬─────────────────┬────────────────────┬─────┐
│ STX │  SeqNo      │   DataLen       │    Command         │ ETX │
│  1B │  3 ASCII    │  4 ASCII        │  Variable ASCII    │  1B │
│0x02 │  "000"–     │  "0001"–        │  (see below)       │0x03 │
│     │  "999"      │  "9999"         │                    │     │
└─────┴─────────────┴─────────────────┴────────────────────┴─────┘
```

> **Important:** The mode-array bytes (M command) are binary, not ASCII. `AioiPacket.ToBytes()` uses Latin-1 encoding to preserve byte values > 0x7F.

Key Commands (AioiEncoder)

Command	String	Description
Initialize	`Z`	Full init; send once at startup
Hardware Reset	`Z1`	Reset all modules (~1.5s delay)

Command	String	Description
Set Mode Array	`M m1[3B] m2[3B] m4[1B]`	Configure LED/SEG/BUZ behavior
Light On (PP1)	`PP1{addr}{qty}`	Instruct module to display quantity
Light Off (D)	`D{addr}`	Extinguish module
Terminate Block	`DB{block2}`	Extinguish all modules in a block (block `00` = all)
Batch Pickup	`P1{block2} {addr4}{qty5}...`	Simultaneous batch instruction (multiple modules)
Batch Overwrite	`P5{block2} {addr4}{qty5}...`	Overwrite in-progress batch
Status Inquiry	`Az{addr}`	Query module status
Scan All	`AZ`	Broadcast status request

3.3 P2L.Core

Business logic layer, hardware-agnostic.

`PickingEngine`

- Central orchestrator for light instructions
- `InstructAsync(instruction)` — sends mode-array + PP1 command to hardware
- `TerminateAsync(address)` — sends D command, updates registry to Standby
- `TerminateAllAsync()` — terminates every active module
- `TerminateBlockAsync(blockNo)` — sends `DB{blockNo}` to extinguish all modules in a block
- Manages `CorrelationId` tracking for batch operations

- Tracks `_pendingAddresses: HashSet<string>` — addresses awaiting confirmation in the current batch
- `IsBatchComplete` → batch active AND no pending addresses remaining
- Fires `BatchCompleted` event when all pending addresses are confirmed

`LightModuleRegistry`

- Thread-safe (`ConcurrentDictionary`) in-memory store of all module states
- `StateChanged: event Action<IReadOnlyList<LightModuleState>>` — fires on any mutation
- `SetInstructed(instruction)` → status = `Operating`
- `SetConfirmed(event)` → status = `Completed` or `Shortage`
- `SetTerminated(address)` → status = `Standby`
- `SetScanned(address, status)` — used by DeviceScanService

`PreemptionGate`

- Controls whether new light commands are accepted
- Used to pause dispatch during CSV task execution

`CsvPickTaskRunner`

- Executes a sequential pick list from a CSV file
- Fires `TaskEvent` for each step: `task_started` , `task_confirmed` , `task_error` , `csv_completed`
- Publishes task events to MQTT via `MqttEventPublisher`

`WatchdogService`

- Monitors the service health
- Restarts sub-components on failure up to `MaxRestarts` within `RestartWindowMinutes`

`RegistryPersistenceService`

- Optional: persists `LightModuleRegistry` state to disk on shutdown / loads on startup
- Enabled via `appsettings.json → EnableStatePersistence: true`

3.4 P2L.Mqtt — MQTT Bridge

The MQTT bridge exposes the P2L service to any MQTT-capable client.

`MqttBridgeService`

- Owns the `IMqttClient` (MQTTnet library)
- Manages TLS connection lifecycle with auto-reconnect
- Subscribes to `CommandTopic` on connect (QoS 1)
- Publishes LWT to `StatusTopic` with `retain=true` on ungraceful disconnect
- **Startup sequence** (first gateway connection only):
  i. Waits for MQTT broker (up to 30s)
  ii. Subscribes to `p2l/devices/#`, collects retained topics for 2s
  iii. Clears all retained device topics + status topic (empty payload, retain=true)
  iv. Runs `DeviceScanService.ScanAsync()` — found devices auto-publish fresh state
  v. Publishes fresh bridge status

`MqttCommandDispatcher`

- Parses incoming JSON from `CommandTopic`
- Routes by `type` field to: `light_on`, `light_off`, `batch_start`, `batch_finish`, `terminate`, `terminate_all`, `csv_start`, `csv_stop`

`MqttEventPublisher`

- Subscribes to `LightModuleRegistry.StateChanged` → publishes `light_changed` to `{DevicesTopic}/{address}` with `retain=true`
- Publishes `confirmed` events to `EventTopic`
- Publishes `batch_completed` to `EventTopic`
- Publishes `bridge_status` to `StatusTopic` with `retain=true`
- Publishes `task_event` to `EventTopic`
- Offline buffer: when broker is unreachable, events are written to `OfflineBufferPath` (JSONL) and replayed on reconnect

`DeviceLabelStore`

- Reads the Python client's SQLite DB (`p2l_devices.db`) at publish time
- Enriches `light_changed` events with the `label` field

P2L.CLI — Operator Commands

Command	Description
`p2l run`	Start the service with full Spectre.Console dashboard
`p2l devices [--start 1] [--end 99]`	Scan address range and list discovered modules
`p2l light on {addr} {qty} [--color Green]`	Light up a module manually
`p2l light off {addr}`	Extinguish a module manually
`p2l terminate-all`	Force-off all active modules

Entry point and operator dashboard.

- **DI wiring** (`Program.cs`): registers all services as singletons, builds `IHost`
- **Spectre.Console dashboard**: live table of module states, log panel
- **Manual override commands**: light on/off, terminate all, trigger scan
- **Configuration**: reads `appsettings.json` from binary directory

---

4. MQTT Protocol Reference

4.1 Topics

Topic	Direction	QoS	Retain	Description
`p2l/commands`	Client → Bridge	1	No	Light control commands
`p2l/events`	Bridge →	1	No	Confirmations, batch events,

Topic	Direction	QoS	Retain	Description
	Client			task events
`p2l/devices/{addr}`	Bridge → Client	1	**Yes**	Per-device state snapshot (4-digit address)
`p2l/status`	Bridge → Client	1	**Yes**	Bridge health + Last Will Testament

> **Retained topics:** Subscribers always receive the latest device state and bridge status immediately on connect, even if no new event has occurred.

4.2 Command Messages (Subscribe on `p2l/commands`)

All commands are JSON objects with a required `type` field and optional `correlationId`.

`light_on` — Light up a module

```
{
  "type": "light_on",
  "correlationId": "optional-uuid",
  "address": "0001",
  "quantity": 5,
  "color": "Green",
  "ledMode": "On",
  "buzMode": null,
  "label": "Shelf A1"
}
```

Field	Type	Required	Default	Values
`address`	string	✅	—	4-digit module address
`quantity`	int	✅	—	Display number (0–99999)
`color`	string	—	`"Green"`	`Off, Red, Green, Blue, Yellow, Cyan, Mage`

Field	Type	Required	Default	Values
`ledMode`	string	—	`"On"`	`On, Off, Flash, HighFlash, NoChange`
`buzMode`	string|null	—	mirrors ledMode	Same as ledMode
`label`	string|null	—	`null`	Device label for enrichment

`light_off` — Extinguish a module

```
{
  "type": "light_off",
  "address": "0001"
}
```

`batch_start` — Start a multi-module picking batch

```
{
  "type": "batch_start",
  "correlationId": "batch-uuid",
  "color": "Green",
  "ledMode": "On",
  "items": [
    { "address": "0001", "quantity": 3, "label": "Shelf A1" },
    { "address": "0002", "quantity": 1, "color": "Red" },
    { "label": "Shelf B3", "quantity": 2 }
  ]
}
```

> Items can be addressed by `address` (4-digit) or by `label` (resolved from the client SQLite DB). Items where neither resolves to a known address are silently skipped.

`batch_finish` — Terminate the active batch

```
{
  "type": "batch_finish",
  "blockNo": "00"
}
```

`terminate` — Force-off a single module

```
{
  "type": "terminate",
  "address": "0001"
}
```

`terminate_all` — Force-off all active modules

```
{
  "type": "terminate_all"
}
```

`csv_start` — Execute a CSV pick list

```
{
  "type": "csv_start",
  "path": "/path/to/tasks.csv"
}
```

`csv_stop` — Cancel running CSV task

```
{ "type": "csv_stop" }
```

4.3 Event Messages (Publish)

`light_changed` — Published to `p2l/devices/{address}` (retained)

Published whenever a module's state changes (lit, extinguished, confirmed, scanned).

```
{
  "type": "light_changed",
  "timestamp": "2026-03-18T03:00:00Z",
  "address": "0001",
  "moduleStatus": "Operating",
  "color": "Green",
  "ledMode": "On",
  "segMode": "On",
  "buzMode": "Off",
  "displayData": "00005",
  "blockNo": "00",
  "adjustedQuantity": null,
  "lastInstructionAt": "2026-03-18T03:00:00Z",
  "lastConfirmedAt": null,
  "label": "Shelf A1"
}
```

`moduleStatus` values	Meaning
`Standby`	Idle, no instruction
`Operating`	Active — lit and waiting for confirm
`Operating2`	Active — alternate state
`Completed`	CONFIRM pressed

`moduleStatus` values	Meaning
`Shortage`	Quantity adjusted (+/- key)
`Maintenance`	Maintenance mode
`Malfunction`	Hardware error

`confirmed` — Published to `p2l/events`

Published when an operator presses CONFIRM on a module.

```
{
  "type": "confirmed",
  "timestamp": "2026-03-18T03:01:00Z",
  "address": "0001",
  "status": "Normal",
  "adjustedQuantity": null,
  "adjustedDisplay": null,
  "correlationId": "batch-uuid",
  "displayData": "00005",
  "label": "Shelf A1"
}
```

`status` values	Meaning
`Normal`	Confirmed as instructed
`Shortage`	Quantity modified via +/- key
`Abnormal`	Error condition

`batch_completed` — Published to `p2l/events`

Published when every module in the current batch has been confirmed.

```
{
  "type": "batch_completed",
  "timestamp": "2026-03-18T03:02:00Z"
}
```

`bridge_status` — Published to `p2l/status` (retained + LWT)

```
{
  "type": "bridge_status",
  "timestamp": "2026-03-18T03:00:00Z",
  "online": true,
  "gatewayConnected": true
}
```

> When the broker receives an **ungraceful disconnect** from the P2L service, it automatically publishes `{ "online": false, "gatewayConnected": false }` as the Last Will Testament.

`task_event` — Published to `p2l/events`

Published during CSV pick-task execution.

```
{
  "type": "task_started",
  "timestamp": "2026-03-18T03:00:00Z",
  "address": "0001",
  "quantity": 5,
  "color": "Green",
  "label": "Item XYZ"
}
```

`type` values	Trigger
`task_started`	Module lit for CSV task
`task_confirmed`	Operator confirmed
`task_error`	Error during task dispatch
`csv_completed`	All CSV tasks finished

---

5. Aioi Hardware Protocol

Reference: *L-Pick Controller Command Reference V4.1*

Connection

- **Protocol:** TCP/IP (Ethernet)
- **Default IP:** `192.168.1.254`
- **Default Port:** `5003`
- **Mode:** Persistent TCP connection; P2L is the client, gateway is the server

Frame Structure

```
STX (0x02) | SeqNo (3 ASCII digits) | DataLen (4 ASCII digits) | Command (ASCII) | ETX (0x
```

- `SeqNo` : rolling 000–999
- `DataLen` : byte length of the Command field
- Latin-1 encoding (preserves binary bytes in mode-array)

Mode Array (M command)

Controls LED/SEG/BUZ behavior for the **next** instruction:

```
M m1 [LED2][LED1+LED0][SEG+BUZ]  m2 [LED2][LED1+LED0][SEG+BUZ]  m4 [0x40]
     └── during instruction ──┘     └── after CONFIRM ─────────┘     └── key config ┘
```

- Each LED state nibble: `Off=1, On=2, Flash=3, HighFlash=4, NoChange=F`
- `LED2` byte: `0x30 | nibble`
- `LED1+LED0` byte: `(LED1_nibble << 4) | LED0_nibble`
- `m4=0x40`: CONFIRM + Fn key enabled

Confirmation Interrupt (`t` packet)

Hardware sends this packet when an operator presses CONFIRM:

```
t {addr4} {status2} {display5}
```

- `addr4`: 4-digit module address
- `status2`: `00`=Normal, `01`=Abnormal, `10`=Shortage
- `display5`: current display value (e.g. `"00005"`)

---

6. Component: P2L Web Client

A Python/FastAPI web application providing a browser-based dashboard and REST API.

```
client/
├── app/
│   ├── main.py          # FastAPI app, WebSocket, MQTT event routing
│   ├── config.py        # AppConfig: loads p2l_config.json / appsettings.json
│   ├── db.py            # SQLite persistence (p2l_devices.db)
│   ├── service.py       # P2L .NET process management (start/stop/detect)
│   ├── core/
│   │   └── registry.py  # In-memory LightModuleRegistry (thread-safe)
│   ├── mqtt/
│   │   └── client.py    # MQTT client (aiomqtt), pub/sub, reconnect loop
│   ├── api/
│   │   ├── routes.py    # REST endpoints
│   │   └── ws_manager.py # WebSocket broadcast manager
│   └── protocol/
│       ├── enums.py     # Python mirrors of LightColor, LightState, ModuleStatus
│       └── models.py    # LightModuleState, PickingInstruction, ConfirmationEvent
├── static/js/app.js     # Browser JS: WebSocket, state, rendering
├── templates/index.html # Single-page dashboard
└── p2l_config.json      # Client configuration
```

REST API Endpoints

Method	Path	Description
`GET`	`/api/status`	Connection status (service + MQTT)
`POST`	`/api/connect`	Connect to MQTT (optionally start P2L service)
`POST`	`/api/disconnect`	Disconnect MQTT
`GET`	`/api/devices`	List all registered devices
`POST`	`/api/devices`	Register a new device
`DELETE`	`/api/devices/{addr}`	Remove a device
`PATCH`	`/api/devices/{addr}`	Update label / order_no
`POST`	`/api/devices/{addr}/light`	Send light_on command

Method	Path	Description
`POST`	`/api/devices/{addr}/off`	Send light_off command
`GET`	`/api/batch/status`	Active batch info
`POST`	`/api/batch/start`	Start batch
`POST`	`/api/batch/finish`	Finish batch
`GET`	`/api/service/status`	P2L service running status
`POST`	`/api/service/start`	Start P2L service
`POST`	`/api/service/stop`	Stop P2L service
`GET`	`/api/config`	Read current config
`PATCH`	`/api/config`	Update config
`GET`	`/api/logs`	Recent log entries
`WebSocket`	`/ws`	Real-time state push

WebSocket Message Types (Server → Browser)

`type`	Payload	Description
`init`	`{connected, mqtt_connected, devices[], host, port}`	Initial state on WS connect
`state_update`	`[LightModuleState, ...]`	Full device list after any registry change
`connection_changed`	`{connected, mqtt_connected, host, port}`	MQTT connect/ disconnect
`bridge_status`	`{online, gateway_connected}`	.NET bridge

`type`	Payload	Description
		health
`confirmation`	`ConfirmationEvent`	Operator pressed CONFIRM
`batch_completed`	`{}`	Batch fully confirmed
`log`	`{time, level, message}`	Service log entry

Device Registry Reconciliation

On each WebSocket connect, the client:

1. Loads devices from SQLite DB → populates in-memory registry
2. Receives MQTT retained `light_changed` messages → updates registry
3. On `bridge_status {online: true, gatewayConnected: true}` (after all retained arrive): removes any registry entry not seen in MQTT retained (stale DB entry for offline hardware)

---

7. Component: SDK

The SDK allows external applications to control P2L via MQTT without dealing with JSON serialization or topic management.

7.1 Python SDK

Location: `tools/sdk/python/p2l.py`
Dependency: `paho-mqtt`
Pattern: Async context manager, callback-based events + async `wait_*` methods

Quick Start

```python
from p2l import P2LClient, LightColor, LightMode

async def main():
    async with P2LClient("broker.host", port=8883, username="u", password="p") as p2l:
        p2l.light_on("0001", quantity=5, color=LightColor.GREEN)
        event = await p2l.wait_confirm("0001", timeout=30)
        print(f"Confirmed: {event.status}, qty={event.adjusted_display}")
```

`P2LClient` Constructor

```python
P2LClient(
    host: str,
    port: int = 8883,
    username: str | None = None,
    password: str | None = None,
    command_topic: str = "p2l/commands",
    event_topic: str = "p2l/events",
    devices_topic: str = "p2l/devices",
    status_topic: str = "p2l/status",
    tls: TLSConfig | None = None,       # None = TLS with OS trust store
    client_id: str = "p2l-sdk-*",
)
```

Control Methods (fire-and-forget, thread-safe)

Method	Description
`light_on(address, quantity, color, led_mode, buz_mode, label)`	Light up a module
`light_off(address)`	Extinguish a module
`batch_start(items, color, led_mode)`	Start multi-module batch
`batch_finish(block_no)`	Finish batch

Async Wait Methods

Method	Description
`await wait_confirm(address, timeout)` → `ConfirmEvent`	Wait for a single module confirmation
`await wait_batch_complete(timeout)`	Wait for `batch_completed` event
`await wait_csv_complete(timeout)`	Wait for CSV task completion

Event Callbacks (register before connecting)

```
p2l.on_confirm(fn)              # fn(ConfirmEvent)
p2l.on_batch_complete(fn)       # fn()
p2l.on_state_change(fn)         # fn(ModuleState) — on any light_changed
p2l.on_bridge_status(fn)        # fn(BridgeStatus)
p2l.on_task_event(fn)           # fn(TaskEvent) — all task events
p2l.on_task_started(fn)         # fn(TaskEvent)
p2l.on_task_confirmed(fn)       # fn(TaskEvent)
p2l.on_task_error(fn)           # fn(TaskEvent)
p2l.on_csv_complete(fn)         # fn()
```

`WaveDispatcher` — Sequential per-address dispatch

Dispatches items one-at-a-time, address by address, waiting for each confirmation before moving to the next.

```
dispatcher = WaveDispatcher(p2l, items=[...], color=LightColor.GREEN)
await dispatcher.start()
stats = await dispatcher.wait(timeout=120)
# stats.confirmed, stats.failed, stats.cancelled
```

`BatchDispatcher` — SDK-managed batch dispatch

Higher-level wrapper around `batch_start` / `batch_finish` with progress tracking.

7.2 Node.js SDK

Location: `tools/sdk/nodejs/p2l.js`
Dependency: `mqtt` npm package
Pattern: Event emitter, callback-based, `async/await` via Promises

Quick Start

```
const { P2LClient, LightColor, LightMode } = require('./p2l');

const p2l = new P2LClient({ host: 'broker.host', port: 8883 });
await p2l.connect();

p2l.lightOn('0001', { quantity: 5, color: LightColor.GREEN });
const event = await p2l.waitConfirm('0001', 30000);
console.log(event);
await p2l.disconnect();
```

Key Classes

Class	Description
`P2LClient`	Main client — connect, control, events
`WaveDispatcher`	Sequential per-address dispatch
`BatchDispatcher`	SDK-managed batch dispatch
`BatchDispatch`	Batch result model

`P2LClient` Methods

Method	Description
`lightOn(address, opts)`	Light up a module
`lightOff(address)`	Extinguish a module
`batchStart(items, opts)`	Start batch

Method	Description
`batchFinish(blockNo)`	Finish batch
`waitConfirm(address, timeoutMs)` → Promise	Wait for confirmation
`waitBatchComplete(timeoutMs)` → Promise	Wait for batch complete
`on('confirm', fn)`	Confirmation event
`on('stateChange', fn)`	Module state change
`on('bridgeStatus', fn)`	Bridge health
`on('batchComplete', fn)`	Batch completed

---

8. Configuration Reference

8.1 P2L appsettings.json

```jsonc
{
  "Gateway": {
    "Host": "192.168.1.254",   // Aioi gateway IP address
    "Port": 5003,              // Aioi gateway TCP port
    "BlockNo": "00"            // Default block number (2-char)
  },
  "LogDirectory": "logs",      // Log file directory (relative to binary)
  "MaxLogFileSizeBytes": 10485760,  // 10 MB log rotation
  "Mqtt": {
    "Enabled": false,          // Enable MQTT bridge
    "Host": "broker.example.com",
    "Port": 8883,              // 8883 = TLS, 1883 = plain
    "ClientId": "p2l-warehouse-01",
    "Username": null,
    "Password": null,
    "CommandTopic": "p2l/commands",
    "EventTopic": "p2l/events",
    "DevicesTopic": "p2l/devices",
    "StatusTopic": "p2l/status",
    "OfflineBufferPath": "mqtt_offline_buffer.jsonl",
    "ClientDbPath": null,      // Path to client's p2l_devices.db for label enrichment
    "Tls": {
      "Enabled": true,
      "CaCertPath": null,      // Custom CA cert (null = OS trust store)
      "ClientCertPath": null,  // Mutual TLS client cert
      "ClientKeyPath": null,   // Separate PEM key file
      "CertPassword": null,
      "AllowUntrustedCerts": false  // DEV ONLY
    }
  },
  "EnableStatePersistence": false,  // Persist registry state across restarts
  "SelfHeal": {
    "Enabled": true,
    "MaxRestarts": 5,              // Max restarts within window
    "RestartWindowMinutes": 10,
    "BackOffSeconds": 3,           // Delay between restarts
    "WatchdogTimeoutMinutes": 5    // Restart if no activity for this long
  },
  "Logging": {
```

```
    "LogLevel": {
      "Default": "Information",
      "P2L.Hardware": "Debug",
      "P2L.Mqtt": "Debug",
      "Microsoft.Hosting": "Warning"
    }
  }
}
```

8.2 Client p2l_config.json

```
{
  "block_no": "00",
  "auto_connect": false,        // Always false — connect via UI button only
  "mqtt": {
    "host": "broker.example.com",
    "port": 8883,
    "client_id": "p2l-webclient",
    "username": null,
    "password": null,
    "command_topic": "p2l/commands",
    "event_topic": "p2l/events",
    "devices_topic": "p2l/devices",
    "status_topic": "p2l/status",
    "tls": {
      "enabled": true,
      "ca_cert_path": null,
      "client_cert_path": null,
      "client_key_path": null,
      "allow_untrusted_certs": false
    }
  },
  "service": {
    "binary_path": "/path/to/publish/p2l",
    "args": [],
    "working_dir": null,        // Defaults to binary directory
    "auto_start": true          // Auto-start P2L service when clicking Connect
  }
}
```

9. Data Flow — End to End

Pick operation (WMS/SDK → hardware → confirmation)

```
1. WMS / SDK / Browser
   POST /api/devices/0001/light  OR  MQTT publish to p2l/commands
   { "type": "light_on", "address": "0001", "quantity": 5, "color": "Green" }

2. P2L.Mqtt — MqttCommandDispatcher
   Deserializes JSON → LightOnCmd
   Calls PickingEngine.InstructAsync(instruction)

3. P2L.Core — PickingEngine
   Updates LightModuleRegistry → status = Operating
   Sends to P2L.Hardware: SetModeForColor(Green) + PP1 command

4. P2L.Hardware — AioiGateway
   Encodes: AioiPacket{ "M m1[...] PP10001 00005" }
   Sends over TCP to Aioi gateway

5. Aioi Gateway → RS-485 → Module 0001
   Module displays "5", LED lights green

6. Operator picks 5 items, presses CONFIRM

7. Aioi Gateway → TCP → P2L.Hardware
   Receives: AioiPacket{ "t 0001 00 00005" }
   Decoded → ConfirmationEvent{ address="0001", status=Normal }

8. P2L.Core — PickingEngine
   LightModuleRegistry.SetConfirmed(event) → status = Completed
   StateChanged fires →

9. P2L.Mqtt — MqttEventPublisher
   Publishes to p2l/devices/0001 (retain):  { "type": "light_changed", "moduleStatus": "Co
   Publishes to p2l/events:                 { "type": "confirmed", "address": "0001", "sta

10. Browser WebSocket receives state_update
    Data Processing tab updates address 0001 row
    If batch: checks if all modules confirmed → batch_completed event
```

10. Startup Sequence

When the P2L service starts and establishes the **first** TCP connection to the Aioi gateway:

```
Gateway connects
       │
       ▼
MqttBridgeService.ConnectionChanged (first time only — Interlocked flag)
       │
       ▼
RunStartupSequenceAsync()
  ├─ Wait for MQTT broker connected (up to 30s)
  ├─ Subscribe p2l/devices/# (2s window)
  ├─ Collect all retained device topics
  ├─ Unsubscribe
  ├─ Clear each retained topic (empty payload, retain=true)
  ├─ Clear p2l/status
  ├─ DeviceScanService.ScanAsync() ──────────────────────────────────┐
  │      Probes addresses 1–99 via Az{addr} command                  │
  │      Found devices → LightModuleRegistry.SetScanned()            │
  │      StateChanged → MqttEventPublisher → p2l/devices/{addr} (retain) │
  └─ Publish bridge_status {online: true, gatewayConnected: true} ◄──┘

Web client receives bridge_status → reconciles registry
  (removes devices in DB but absent from MQTT retained)
```

On subsequent gateway reconnects: only bridge status is updated. No re-scan or re-clear.

---

11. Build & Deployment

Local Build (macOS/Linux)

```
./deploy/package.sh [version]   # Builds for current platform + Linux (Docker)
```

Outputs to `dist/`:

- `p2l-linux-x64-{ver}.tar.gz`
- `p2l-linux-arm64-{ver}.tar.gz`
- `p2l-osx-arm64-{ver}.tar.gz`
- `p2l-osx-x64-{ver}.tar.gz`

GitHub Actions — Cross-Platform CI

Workflow: `.github/workflows/build-all.yml`
Trigger: push to `main`, pull request, or `v*` tag

Job	Runner	Output
`linux-x64`	`ubuntu-24.04`	AOT binary + tar.gz
`linux-arm64`	`ubuntu-24.04-arm`	AOT binary + tar.gz
`osx-arm64`	`macos-latest`	AOT binary + tar.gz
`osx-x64`	`macos-13`	AOT binary + tar.gz
`windows-x64`	`windows-latest`	AOT binary + PyInstaller exe + **WiX 5 MSI**
`release`	—	GitHub Release with all artifacts (on `v*` tag)

Windows MSI Installer

- **Tool:** WiX Toolset v5 (`dotnet tool install --global wix --version 5.0.2`)
- **Project:** `deploy/P2L.Installer/P2L.Installer.wixproj`
- **Contents:**
  - `p2l.exe` — .NET AOT service binary
  - `p2l-client.exe` — Python web client (PyInstaller bundle)
  - `appsettings.json` — default configuration
  - Windows Service registration (P2L Service + P2L Client)
  - Shortcuts in Start Menu

Linux systemd Service

```
[Unit]
Description=Pick2Light Service
After=network.target

[Service]
ExecStart=/opt/p2l/p2l
WorkingDirectory=/opt/p2l
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Native AOT Notes

- No .NET runtime required on target machine
- Single-file executable (~15–25 MB)
- AOT restrictions: no `System.Reflection.Emit`, JSON uses Source Generators (`P2LMqttJsonContext`)
- macOS AOT requires OpenSSL + Brotli via Homebrew (`LIBRARY_PATH` set in build script)